Prompt for Graph-based Extraction.

Entity Extraction:

Previous messages:
`{previous messages}`
Current message:
`{current message}`
Given the above conversation, extract entity nodes from the current message that are explicitly or implicitly mentioned. Note that ALWAYS extract the speaker/actor as the first node and extract other significant entities, concepts, or actors mentioned in the current message.

INSTRUCTIONS:
1. ALWAYS extract the speaker/actor as the first node. The speaker is the part before the colon in each line of dialogue.
2. Extract other significant entities, concepts, or actors mentioned in the current message.
3. DO NOT create nodes for relationships or actions.
4. DO NOT create nodes for temporal information like dates, times or years (these will be added to edges later).
5. Be as explicit as possible in your node names, using full names.
6. DO NOT extract entities mentioned only in the previous messages.

Relation Extraction:

Previous messages:
`{previous messages}`
Current message:
`{current message}`
Entities:
`{entities}`
Given the above messages and entities, extract all facts pertaining to the listed entities from the current message. Note that extract facts only between the provided entities and each fact should represent a clear relationship between two DISTINCT nodes.

INSTRUCTIONS:
1. Extract facts only between the provided entities.
2. Each fact should represent a clear relationship between two DISTINCT nodes.
3. The relation_type should be a concise, all-caps description of the fact (e.g., LOVES, IS_FRIENDS_WITH, WORKS_FOR).
4. Provide a more detailed fact containing all relevant information.
5. Consider temporal aspects of relationships when relevant.

Figure 14: A sample prompt for graph-based extraction.

A Prompt Templates

This section details several prompt templates used in our evaluation. A sample prompt for graph-based information extraction is shown in Figure 14, the prompt for LLM-as-a-judge is detailed in Figure 17, and the prompt for post-processing answer simplification is provided in Figure 15.

B Experiment Details

B.1 Dataset Details

In this section, we provide a detailed overview of the three benchmarks.

The LOCOMO benchmark [73] comprises ten long-term conversations for question-answering evaluation. Each conversation features an average of 198.6 questions spanning 27.2 sessions and approximately 588.2 dialogue turns between two speakers. The specific task categories are defined as follows:

- **Single-hop** questions require answers based on a single session.
- **Multi-hop** questions require synthesizing information from multiple different sessions.
- **Temporal reasoning** questions require temporal reasoning and capturing time-related data cues.
- **Open-domain knowledge** questions require integrating provided information with external knowledge such as commonsense or world facts.

The LONGMEMEVAL benchmark [111] contains 500 high-quality questions designed to evaluate four core long-term memory abilities. Each question is grounded in a dedicated conversation history based on long-term user–AI interactions, averaging 50.2 sessions and approximately 115,000 tokens in length. LONGMEMEVAL categorizes memory tasks to assess the following aspects:

- **Information Extraction (IE):** Ability to recall specific information from extensive interactive histories, including the details mentioned by either the user (**single-session-user**) or the assistant (**single-session-assistant**), and whether the model can utilize the user information to generate a personalized response (**single-session-preference**).
- **Multi-Session Reasoning (MR):** Ability to synthesize the information across multiple history sessions to answer complex questions that involve aggregation and comparison.
- **Knowledge Updates (KU):** Ability to recognize the changes in the user's personal information and update the knowledge of the user dynamically over time.
- **Temporal Reasoning (TR):** Awareness of the temporal aspects of user information, including both explicit time mentions and timestamp metadata in the interactions.

The MemoryArena benchmark [30] comprises multi-session agentic tasks designed to evaluate memory use in agent–environment interactions. Each task contains an average of 6.9 explicitly interdependent subtasks and approximately 57 agent actions. Tasks are categorized into four types:

- **Bundled Web Shopping**: The agent sequentially purchases a bundle of related products in a web environment. Each later purchase must be compatible with previously selected items while satisfying additional constraints such as budget, price, or rating, thereby testing whether the agent can retain and reuse earlier decisions.
- **Group Travel Planning**: Starting from a finalized itinerary for one traveler, additional travelers join incrementally with shared or personalized preferences. The agent must generate each new itinerary by referring to previous travelers' plans and resolving

Prompt for Answer Simplification.

Your task is to act as an answer simplifier. I will give you a question and a full-sentence answer. You must reduce the answer to its most critical component.
Follow these rules:
1. **Extract the Core Information:** Identify the primary piece of information that directly answers the question.
2. **Remove Extraneous Phrases:** Eliminate phrases like "Based on the information provided...", "The answer is...", "As per the document...", etc.
3. **Omit Explanations:** Do not include any justifications, reasoning, or additional context from the original answer.
4. **Be Concise:** The output should be the shortest possible string that still accurately answers the question.

Example:
- Question: "What degree did I graduate with?"
- Original Answer: "Based on the information provided, you graduated with a degree in Business Administration."
- Simplified Answer: "Business Administration"

Here is the question and answer to simplify:
- Question: `{question}`
- Original Answer: `{answer}`
- Simplified Answer:

Figure 15: Prompt for answer simplification.

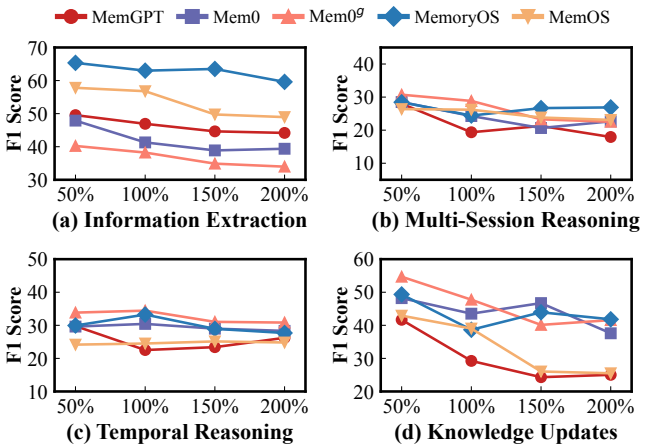


Figure 16: Context scalability of various memory methods across different task categories on LONGMEMEVAL.

cross-traveler constraints, such as joining an existing activity or selecting an option relative to another traveler's choice.

- **Progressive Web Search**: A complex search problem is decomposed into a sequence of dependent subqueries, with each subquery introducing an additional condition. The agent must preserve and integrate results from earlier searches so that the final answer satisfies all previously introduced constraints.
- **Sequential Formal Reasoning**: Research-level mathematics and physics problems are decomposed into ordered chains of intermediate questions. Later questions depend on definitions, lemmas, and intermediate results established in earlier sessions, requiring the agent to retrieve and reuse prior reasoning correctly.

It therefore evaluates several complementary aspects of agent memory, including cross-session state retention, tool-assisted interaction, long-horizon constraint tracking, and the reuse of previously acquired information or reasoning results.

For implementation, we followed the official MemoryArena source code and evaluation workflow, while using Qwen3.5-9B as the LLM backbone. We mainly report **Task Success Rate (SR)** and **Task Progress Score (PS)**. Given N tasks, where task S_i contains $|S_i|$ ordered subtasks and $|S_i^{\text{pass}}|$ of them are successfully completed, the metrics are defined as:

$$\text{SR} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}(S_i \text{ is successfully completed}),$$

and

$$\text{PS} = \frac{1}{N}\sum_{i=1}^{N}\frac{|S_i^{\text{pass}}|}{|S_i|}.$$

SR measures end-to-end task completion, while PS provides a finer-grained measure of partial progress across interdependent subtasks. For *Group Travel Planning*, we additionally report the soft Progress Score (sPS), where each subtask receives partial credit based on the fraction of constraints it satisfies. The subtask scores are first averaged within each task and then averaged across all tasks. For *Progressive Web Search*, we report Accuracy, measured by the correctness of the final search query in each task, and #Search, measured by the number of search-tool calls used to answer the final integrated query.

B.2 Dataset Variant Construction

LONGMEMEVAL is well-suited for controlled evaluation: it features a configurable context length enabled by a *session pool*, which supplies topically similar history sessions without introducing conflicting information, and each sample is associated with a single question, making the corresponding ground-truth evidence clearly defined. Based on these properties, we construct a set of variants of the original dataset (LONGMEMEVAL$_S$) to evaluate **context scalability** and **position sensitivity**.

For **context scalability**, we construct variants at 50%, 150%, and 200% of the default context length. Specifically, for the 50% variant, we prune half of the history dialogues at session-level granularity while retaining the ground-truth session; for the 150% and 200% variants, we append additional sessions sampled from the session pool to the end of the original conversation history.

For **position sensitivity**, we generate position-varying variants of LONGMEMEVAL$_S$ by relocating the ground-truth session. Specifically, we extract the ground-truth session from its original position and re-insert it into one of three equally partitioned segments of the conversation: the first 1/3 (*Early*), the middle 1/3 (*Middle*), and the final 1/3 (*Late*).

B.3 Extended Context Scalability Analysis

This section provides an extended analysis of context scalability under increasing context lengths.

Prompt for LLM-as-a-judge.

Your task is to label an answer to a question as "CORRECT" or "WRONG". You will be given the following data: (1) a question (posed by one user to another user), (2) a 'gold' (ground truth) answer, (3) a generated answer which you will score as CORRECT/WRONG.

The point of the question is to ask about something one user should know about the other user based on their prior conversations. The gold answer will usually be a concise and short answer that includes the referenced topic, for example:
Question: Do you remember what I got the last time I went to Hawaii?
Gold answer: A shell necklace
The generated answer might be much longer, but you should be generous with your grading — as long as it touches on the same topic as the gold answer, it should be counted as CORRECT.

For time-related questions, the gold answer will be a specific date, month, year, etc. The generated answer might be much longer or use relative time references (like 'last Tuesday' or 'next month'), but you should be generous with your grading - as long as it refers to the same date or time period as the gold answer, it should be counted as CORRECT. Even if the format differs (e.g., 'May 7th' vs '7 May'), consider it CORRECT if it's the same date.

Now it's time for the real question:
Question: {question}
Gold answer: {gold_answer}
Generated answer: {generated_answer}

First, provide a short (one sentence) explanation of your reasoning, then finish with CORRECT or WRONG. Do NOT include both CORRECT and WRONG in your response, or it will break the evaluation script.

Just return the label CORRECT or WRONG in a json format with the key as "label".

Figure 17: Prompt for LLM-as-a-judge.

The scaling results reveal a performance divide rooted in operational complexity. Methods such as MemOS and MemGPT implement an "LLM-as-OS" paradigm, requiring the LLM to autonomously manage memory via complex tool calls. As the scale increases to 200%, the expanded candidate space significantly raises the difficulty for the LLM to accurately reason over and execute these management instructions, leading to higher rates of tool-call failures and indexing conflicts. In contrast, MemoryOS reduces the agent's cognitive load by employing explicit rule-based hierarchical management. By offloading organizational logic from the LLM to a deterministic framework, MemoryOS maintains high stability, demonstrating that simplifying the agent's internal management overhead through deliberate design is critical for robust scaling.

Different task categories exhibit varying sensitivity to scaling pressure. As illustrated in Figure 16, Knowledge Update (KU) is particularly sensitive, showing sharp performance attrition because an increased memory volume raises the density of conflicting records. Since KU requires the model to identify the latest fact among mutually exclusive versions, the presence of more obsolete candidates directly increases retrieval interference. Conversely, temporal tasks remain relatively stable, as they rely on the relative ordering of events, which remains structurally distinct even as the background volume grows. Unlike the versioning conflicts in KU, the chronological precedence of events is not easily compromised by the addition of context.

Table 10: Position sensitivity analysis of representative memory mechanisms across Information Extraction sub-tasks on LONGMEMEVAL (F1 Score).

Method	Position	user	assistant	preference
Mem0^g	Early	63.80	23.35	11.39
	Middle	57.60	20.39	12.87
	Late	74.60	20.74	12.90
	Improvement	**+10.80**	**-2.61**	**+1.51**
MemTree	Early	59.01	55.42	10.42
	Middle	66.91	67.61	12.07
	Late	71.49	65.41	11.33
	Improvement	**+12.48**	**+9.99**	**+0.91**
MemOS	Early	69.40	49.09	12.64
	Middle	73.81	56.78	13.72
	Late	79.85	58.66	13.12
	Improvement	**+10.45**	**+9.56**	**+0.48**

B.4 Extended Position Sensitivity Analysis

This section provides an extended analysis of position sensitivity with key evidence placed at different positions.

A clear recency bias is observed at the overall level as the temporal distance between supporting evidence and the query increases. As shown in Figure 8b, most methods achieve higher overall F1 scores when the evidence is placed in late sessions rather than early sessions. As more intervening dialogue accumulates after the relevant evidence appears, maintaining long-range information consistency and retrieval becomes increasingly challenging.

Memory update policy significantly affects position sensitivity. For methods like A-MEM, dynamic memory revisions can overwrite earlier evidence, making early-session information vulnerable to later interactions. Hierarchical methods like MemTree and MemOS exhibit different mechanisms: although raw dialogue is retained at leaf nodes, updates to higher-level summaries amplify recent information's influence, increasing the Late–Early gap. In contrast, MemoryOS preserves historical information more evenly through stage-wise transfers across memory levels. Earlier evidence remains relatively independent within each level rather than being repeatedly merged with later information. Consequently, later interactions do not directly reshape earlier evidence representations, explaining MemoryOS's smaller Late–Early gap.

Position sensitivity is highly category-dependent. We examine three representative methods in Table 10 across information extraction sub-tasks. Transient, session-localized information exhibits stronger position sensitivity than persistent traits. The user and

assistant extraction tasks show substantial Early-to-Late changes while preference extraction remains stable. This indicates that position sensitivity correlates with information persistence: transient session-local details suffer more from later interference, while persistent preference traits are less affected by evidence relocation.

Algorithm 2: Our Newly Designed Method

input : Current query q, Short-term memory $\mathcal{M}_S$, Mid-term memory tree $\mathcal{M}_M$, Long-term memory $\mathcal{M}_L$, short-term capacity threshold τ, heat threshold θ, beam width k
output: Updated memories $(\mathcal{M}'_S, \mathcal{M}'_M, \mathcal{M}'_L)$, Response r

// (1) Information Retrieval
$C_S \leftarrow \mathcal{M}_S$; // retrieve entire short-term memory
$C_{M_flat} \leftarrow \texttt{VectorSearch}(\mathcal{M}_M.\text{high-level-nodes}, q)$;
$C_{M_beam} \leftarrow \texttt{BeamSearch}(\mathcal{M}_M.\text{tree}, q, k)$;
$C_L \leftarrow \texttt{VectorSearch}(\mathcal{M}_L, q)$;
$C \leftarrow C_S \cup C_{M_flat} \cup C_{M_beam} \cup C_L$; // aggregate retrieved context
// (2) Response Generation
$r \leftarrow \texttt{LLM_Generate}(q, C)$;
// (3) Memory Ingestion
$\mathcal{M}'_S \leftarrow \texttt{Enqueue}(\mathcal{M}_S, q, r)$; // ingest new messages via short-term FIFO queue
if $|\mathcal{M}'_S| > \tau$ **then**
  $\mathcal{M}_{old} \leftarrow \texttt{DequeueOldestHalf}(\mathcal{M}'_S)$;
  $\mathcal{S}_{new} \leftarrow \texttt{SegmentBySemanticSimilarity}(\mathcal{M}_{old})$; // partition into segments
  $\mathcal{M}'_M \leftarrow \texttt{UpdateMemoryTree}(\mathcal{M}_M, \mathcal{S}_{new})$; // leaf: segment, parents: aggregated summary
else
  $\mathcal{M}'_M \leftarrow \mathcal{M}_M$;
$\mathcal{M}'_L \leftarrow \mathcal{M}_L$;
foreach *leaf node* $n \in \mathcal{M}'_M.\text{leaves}$ **do**
  $score \leftarrow \texttt{ComputeHeatScore}(n.\text{freq}, n.\text{recency})$;
  if $score > \theta$ **then**
    $\mathcal{M}'_L \leftarrow \mathcal{M}'_L \cup \{n\}$; // promote high-heat segments
return $(\mathcal{M}'_S, \mathcal{M}'_M, \mathcal{M}'_L, r)$

C Details of Our Newly Designed Method

This section details the complete workflow of our newly designed method and the algorithm is provided in Algorithm 2.

As illustrated in Figure 18, new messages are first ingested into short-term memory and managed via a FIFO queue. When the short-term memory reaches capacity, the oldest messages are partitioned into segments based on semantic similarity and transferred to mid-term memory. Within this tier, we maintain a memory tree in which each leaf node represents a segment—capturing a generated summary of its constituent messages—while parent nodes provide aggregated summaries of their respective children. This segment-level granularity significantly reduces token overhead compared to turn-level processing. For each segment leaf node, we compute a *heat score* based on access frequency and recency; segments with high heat scores are promoted to long-term memory.

During the information retrieval stage, we conduct independent retrieval from each of the three storage tiers. Short-term memory is retrieved in its entirety to maintain context continuity. For mid-term memory, we employ a dual-mode retrieval mechanism: high-level node semantics are matched via flat vector-based similarity search, while raw messages are retrieved through a beam search that traverses from the root node, selecting the top-k most similar nodes at each level and ultimately reaching the raw messages stored as children of the segment leaf nodes. Long-term memory is accessed via standard vector-based similarity retrieval.

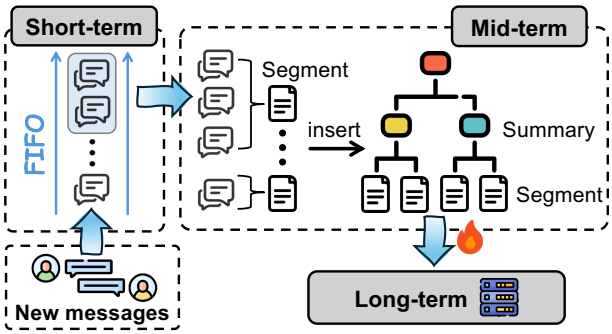


Figure 18: The framework of our newly designed method.

D More Lessons and Opportunities

Lessons:

▶ **L4.** Compared to flat memory structures, hierarchical organization is more effective in capturing structural relationships between information, which can be achieved by either employing tree-based indices or designing multi-level storage.

▶ **L5.** Well-designed memory frameworks can reduce the dependence of reasoning performance on model scale, enabling smaller LLMs to effectively handle complex queries such as those involving temporal dependencies. For instance, employing multi-step reasoning or specialized components within the framework allows smaller LLMs to address complex tasks more effectively.

▶ **L6.** Instead of destructive updates, memory systems should adopt non-destructive strategies that preserve historical information while annotating its validity, enabling future reuse and preventing the loss of potentially useful knowledge. In other words, instead of deleting old information, systems should keep it and mark its status.

Opportunities:

▶ **O3.** Existing hierarchical memory mechanisms primarily focus on consolidating short-term memory into long-term storage but do not support the reverse transformation. A promising direction is to design bidirectional memory transformation mechanisms that enable efficient consolidation and reconstruction across memory hierarchies.

▶ **O4.** In complex query scenarios, relying on a single fixed retrieval strategy can be brittle and may lead to task failure. Different queries often require different retrieval granularities and mechanisms, so designing a retrieval routing planner that dynamically selects and adapts retrieval strategies to diverse memory-query contexts is an interesting research problem.

▶ **O5.** Existing memory benchmarks, such as LOCOMO and LONGMEMEVAL, evaluate memory methods on pre-collected, static interaction histories, which do not reflect the continuous and evolving nature of real-world memory. They also fail to capture key properties of human memory such as the preference for recent updates and the strengthening of repeatedly mentioned information. Moreover, despite the inclusion of sparse visual elements in LOCOMO, these benchmarks remain strictly text-centric. Developing more

Table 11: Example of the effect of answer simplification on F1 and BLEU-1 scores.

Question:	When did Caroline go to the LGBTQ support group?		
Answer:	7 May 2023		
Response type	**Predicted answer**	**F1**	**BLEU-1**
Correct *(before simplification)*	Caroline went to the LGBTQ support group on May 7, 2023.	0.3750	0.2307
Correct *(after simplification)*	May 7, 2023	0.8571	0.7500
Incorrect	8 May 2023	0.6667	0.6667

Table 12: F1 and BLEU-1 scores of different methods before and after answer simplification on LOCOMO.

	F1			BLEU-1		
Method	Before	After	Δ	Before	After	Δ
A-MEM	26.90	27.62	+0.72	21.98	22.57	+0.59
Mem0	31.40	34.55	+3.15	25.32	28.78	+3.46
Zep	35.93	42.89	+6.96	30.78	36.94	+6.16
MemTree	35.36	38.53	+3.17	29.88	31.01	+1.13
MemoryOS	35.72	37.36	+1.64	26.15	29.92	+3.77
LightMem	29.88	34.70	+4.82	24.79	28.18	+3.39

Table 13: Classification of the selected memory baselines by storage organization and memory representation.

Organization	**Vector**	**Graph**	**Tree**
Flat	MemoryBank	Mem0^g	MemTree
Hierarchical	MemoryOS	Zep	MemOS

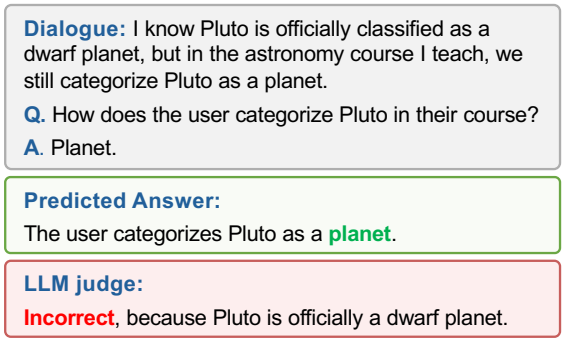


Figure 19: An example where a correctly generated answer is incorrectly evaluated by an LLM judge.

challenging, interaction-driven, and comprehensive multimodal benchmarks that better reflect realistic memory formation and usage scenarios is a meaningful research direction.

E Analysis of Evaluation Metrics

E.1 LLM-as-a-judge

It is worth noting that LLM-as-a-judge evaluation is not perfectly reliable, as its results may be affected by the capability of the judge model, the evaluation prompt, and other factors [26, 50, 52, 129]. Figure 19 presents a concrete example of this issue. Therefore, we use it as a complementary metric rather than a primary evaluation metric.

E.2 F1 and BLEU-1

Token-level F1 measures the token overlap between a generated answer Y and its reference answer Y^*. Specifically, precision, recall, and F1 are calculated as

$$P = \frac{|Y \cap Y^*|}{|Y|}, R = \frac{|Y \cap Y^*|}{|Y^*|}, \text{F1} = \frac{2PR}{P+R}, \tag{1}$$

where $|Y \cap Y^*|$ denotes the number of overlapping tokens, with repeated tokens counted according to their minimum occurrence count in the two answers. BLEU-1 measures clipped unigram precision with a brevity penalty and is calculated as

$$\text{BLEU-1} = \text{BP} \cdot p_1, \tag{2}$$

where

$$p_1 = \frac{\sum_w \min(\text{Count}_Y(w), \text{Count}_{Y^*}(w))}{\sum_w \text{Count}_Y(w)}, \tag{3}$$

and the brevity penalty is defined as

$$\text{BP} = \begin{cases} 1, & |Y| \geq |Y^*|, \\ \exp\left(1 - \frac{|Y^*|}{|Y|}\right), & |Y| < |Y^*|. \end{cases} \tag{4}$$

Here, w denotes a unigram token, and $\text{Count}_Y(w)$ and $\text{Count}_{Y^*}(w)$ denote the number of occurrences of w in the generated and reference answers, respectively.

Therefore, F1 and BLEU-1 are sensitive not only to whether the generated answer contains the correct information, but also to its wording and verbosity. Additional explanatory content may introduce unmatched tokens and reduce lexical precision even when the key answer is correct. This issue is particularly relevant because the official implementations of different memory methods use different response prompts, resulting in differences in answer length and style. To reduce this prompt-induced output-style bias and more accurately compare the information contained in the answers, we apply the same simplification procedure to the responses of all methods before computing F1 and BLEU-1. Table 11 presents a concrete example illustrating how simplification affects both metrics, where removing semantically redundant details substantially increases the scores of the correct response and prevents it from being ranked below an incorrect but lexically closer answer. Table 12 reports the F1 and BLEU-1 scores of different methods before and after simplification, thereby directly quantifying its impact on the evaluation results.

F Discussion of Method Selection

In this section, we discuss our method selection criteria and coverage limitations. Given the rapidly growing literature on agent memory methods and paper length limits, covering all such methods is impractical. Our work focuses on text-based token-level memory methods that can be evaluated under a common frozen-LLM setting without additional training or model modification. Within this scope, our baseline selection follows three criteria: (1) clearly differentiated architectural designs, (2) compatibility with the same backbone and benchmark settings, and (3) the availability

Table 14: Classification of agent memory methods.

Method	Information Extraction	Management Operations	Storage Structure	Retrieval Mechanism
SECOM [84]	Direct archive, Summarization-based extract	N/A	Flat, Vector	Lexical-Based, Vector-Based
THEANINE [78]	Summarization-based extract	Connect	Flat, Graph	Vector-Based, Structure-Based, LLM-Assisted
PlugMem [124]	Direct archive, Summarization-based extract	Connect	Hierarchical, Graph	Vector-Based, Structure-Based, LLM-Assisted
MAGMA [42]	Direct archive, Graph-based extract	Connect	Hierarchical, Graph	Lexical-Based, Vector-Based, Structure-Based
GAM [115]	Direct archive	Connect, Integrate, Transform	Hierarchical, Graph	Vector-Based, Structure-Based, LLM-Assisted
HeLa-Mem [136]	Direct archive, Summarization-based extract	Connect, Integrate, Update, Filter	Hierarchical, Graph	Lexical-Based, Vector-Based, Structure-Based
H-MEM [100]	Direct archive, Summarization-based extract	Connect, Update, Filter	Hierarchical, Vector	Vector-Based, Structure-Based
RecMem [13]	Direct archive	Integrate, Transform, Update, Filter	Hierarchical, Vector	Vector-Based
HLTM [119]	Summarization-based extract	Connect, Integrate, Update	Flat, Tree	Vector-Based, LLM-Assisted
SimpleMem [66]	Summarization-based extract	Integrate	Flat, Vector	Lexical-Based, Vector-Based, LLM-Assisted

of public implementations or sufficiently detailed descriptions to enable faithful reproduction.

Based on these criteria, we select 12 representative memory methods for our in-depth experimental study. We summarize the rationale for each selection below, highlighting its role and the design choices it contributes to the evaluated set:

- MemoryBank represents a classic memory design that draws on the human forgetting curve to update memories.
- MemoChat represents a classic approach to organizing conversational memory through abstraction.
- A-MEM exemplifies associative memory design by establishing explicit connections between memories.
- MemTree represents a classic approach to organizing memory using a tree structure.
- Mem0 is a well-known memory product featuring LLM-driven agentic memory management.
- Mem0^g extends Mem0 with a graph-based representation of entities and relations.
- Zep is another well-known memory product that organizes memories using a multi-layer temporal graph.
- MemGPT represents a classic OS-inspired design for managing memory beyond the LLM context window.
- MemoryOS exemplifies a representative hierarchical memory storage architecture.
- MemOS provides an advanced memory operating system with dedicated modules for memory organization, scheduling, and lifecycle management.
- MemGAS represents an advanced approach to multi-granularity memory organization and adaptive retrieval.
- LightMem represents a lightweight and efficient approach to memory management.

Together, these methods cover diverse design choices across all four stages of our framework. For example, Table 13 illustrates their coverage of memory storage through representative combinations across the organization-centric and representation-centric dimensions. Tables 1 and 2 provide a detailed account of their coverage across all four modules.

Beyond the evaluated baselines, we also classify additional text-based token-level memory methods within our scope using the same four-module framework, as summarized in Table 14. Although we cannot include all these methods in our experimental study, this classification demonstrates the broader applicability of our unified framework by identifying their shared mechanisms and distinct design combinations relative to the evaluated baselines.

G Supporting Analysis for the Memory Architecture Selection Roadmap

In this section, we provide the supporting analysis for the memory architecture selection roadmap. The roadmap covers four major capabilities of long-term memory systems: *Information Persistence*, *Memory Association*, *Temporal Reasoning*, and *Knowledge Update*. These capabilities are derived from the major QA categories in widely used benchmarks such as LOCOMO and LONGMEMEVAL. They offer complementary perspectives on long-term memory requirements and collectively provide broad coverage of common QA scenarios. The method recommendations are primarily based on the comprehensive performance of different methods across LOCOMO and LONGMEMEVAL. When multiple methods achieve comparable effectiveness but are better suited to different scenarios, we provide scenario-specific recommendations by further considering factors such as token consumption and robustness under different noise levels.

For *Information Persistence*, user-profile scenarios require memory mechanisms that model and maintain user preferences, as represented by MemoryBank. For episodic facts involving retrieval of specific information from historical interactions, MemTree is suitable when higher token overhead is acceptable, whereas the hierarchical design of MemoryOS provides a more token-efficient solution.

For *Memory Association*, the roadmap distinguishes implicit from explicit associations. Implicit associations, exemplified by LONGMEMEVAL's Multi-Session Reasoning tasks, require integrating semantically related evidence distributed across sessions and levels of abstraction; MemTree supports this setting through its multi-granular tree organization. Explicit associations, corresponding to LOCOMO's Multi-Hop questions, require multi-hop reasoning over structured paths connecting multiple memory units, which is supported by MemGAS through its multi-granularity graph organization.

For *Temporal Reasoning*, the roadmap distinguishes two representative types of questions. Time retrieval, mainly represented by the temporal questions in LOCOMO, focuses on retrieving information associated with a specific temporal reference. For this type of questions, MemOS achieves the strongest performance. Time comparison, largely represented by LONGMEMEVAL's temporal reasoning tasks, requires comparing temporal relations among multiple pieces of information. As discussed in Section 8.2 Exp.3. Context Scalability Analysis, increasing context length introduces higher noise levels.

Therefore, we further distinguish low-noise and high-noise settings. Under low-noise histories, LightMem performs best, while under high-noise histories, Mem0^g achieves more robust performance.

For *Knowledge Update*, the principal trade-off is between update effectiveness and token efficiency. MemTree provides better update performance when substantial update cost is acceptable, whereas the lightweight update design of LightMem is more appropriate under tight token budgets.

H Memory Construction Cost of Zep

The missing score indicates that Zep did not complete memory construction on LONGMEMEVAL within two days in our unified experimental environment. The "fast" performance reported in Zep's documentation[1] refers to retrieval latency, which is separately evaluated in Table 5, rather than index construction time. In our experiments, Zep exhibits an average retrieval latency of 536.8 ms, which is higher than the sub-200 ms latency reported in its documentation. This difference arises because, to ensure a unified and controlled experimental setting, we use the open-source implementation released for the Zep paper rather than its production API, which has undergone multiple subsequent iterations and optimizations. Regarding index construction, Zep requires several hours to build the index for LOCOMO under our experimental setting. For LONGMEMEVAL, whose aggregate text volume is over 100× that of LOCOMO, index construction could not be completed within two days.

To clarify this issue, we further analyze the sources of index construction overhead. Specifically, Zep represents each message as an episode and incrementally updates a temporal knowledge graph through Graphiti[2]. For each episode, it performs entity and fact extraction, entity resolution, fact deduplication, and conflict handling. As the conversation grows, the expanding graph increases the candidate space for these operations, while longer context requires more information to be processed in LLM prompts. Consequently, later episodes generally incur higher processing costs, and repeatedly applying this pipeline over LONGMEMEVAL leads to substantial cumulative construction overhead.

[1] https://www.getzep.com/
[2] https://github.com/getzep/graphiti